APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 10: Redes Neuronales Recurrentes

19 de octubre de 2026

Índice general

1 Apuntes del Tema 10: Redes Neuronales Recurrentes **2**

1.1 1. De Datos Estáticos a Datos Secuenciales 2

1.1.1 1.1 Limitaciones de los Modelos Feedforward (MLPs y CNNs) 2

1.1.2 1.2 Modelos Autorregresivos y el Estado Oculto 2

1.2 2. Arquitectura de la Red Neuronal Recurrente Básica (RNN Vanilla) 2

1.2.1 2.1 Ecuaciones del Estado Oculto y Salida 2

1.3 3. Entrenamiento: Retropropagación a través del Tiempo (BPTT) 3

1.3.1 3.1 Backpropagation Through Time (BPTT) 3

1.3.2 3.2 Problema Crítico: Desvanecimiento y Explosión del Gradiente 4

1.4 4. Redes Gated: LSTM y GRU 4

1.4.1 4.1 LSTM (*Long Short-Term Memory*) 4

1.4.2 Puertas y Ecuaciones Matemáticas de la LSTM: 4

1.4.3 4.2 GRU (*Gated Recurrent Unit*) 5

1.5 5. RNNs Bidireccionales y Apiladas (*Deep RNNs*) 5

1.6 6. Práctica y Aplicación en Código (PyTorch) 6

1 Apuntes del Tema 10: Redes Neuronales Recurrentes

1.1 1. De Datos Estáticos a Datos Secuenciales

1.1.1 1.1 Limitaciones de los Modelos Feedforward (MLPs y CNNs)

- **Asunción de Muestras Independientes (*i.i.d.*):** Los modelos tradicionales como los Perceptrones Multicapa (MLP) o las Redes Convolucionales (CNN) asumen que las muestras de entrada son independientes e idénticamente distribuidas.
- **Entradas de Tamaño Fijo:** Un MLP o una CNN requieren un vector o tensor de dimensiones de entrada rígidamente predefinidas.
- **Incapacidad para Capturar Dependencias Temporales:** En datos con orden temporal o secuencial (como texto, audio, series temporales financieras o trayectorias GPS), la información del elemento actual depende de los elementos anteriores.

1.1.2 1.2 Modelos Autorregresivos y el Estado Oculto

Para modelar secuencias $X = (x_1, x_2, \ldots, x_T)$, deseamos estimar la probabilidad conjunta mediante la regla de la cadena de probabilidades:

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- **Modelos de Marcov (Orden k):** Asumen que x_t solo depende de los k pasos anteriores.
- **Mecanismo de Estado Oculto (*Hidden State*): **En lugar de truncar la historia a k pasos, una Red Neuronal Recurrente (RNN) mantiene un** estado latente h_t** que actúa como una memoria comprimida de toda la secuencia procesada desde $t = 1$ hasta $t - 1$.

1.2 2. Arquitectura de la Red Neuronal Recurrente Básica (RNN Vanilla)

1.2.1 2.1 Ecuaciones del Estado Oculto y Salida

En cada paso de tiempo t, la RNN recibe la entrada actual $x_t \in \mathbb{R}^d$ y el estado oculto del paso anterior $h_{t-1} \in \mathbb{R}^h$.

1. **Actualización del Estado Oculto:**

$$h_t = \phi\left(x_t W_{xh} + h_{t-1} W_{hh} + b_h\right)$$

1. **Cálculo de la Salida:**

$$\hat{y}_t = \text{Softmax}\left(h_t W_{hq} + b_q\right) \quad \text{o} \quad o_t = h_t W_{hq} + b_q$$

donde:

- $W_{xh} \in \mathbb{R}^{d \times h}$: Matriz de pesos para la entrada.

- $W_{hh} \in \mathbb{R}^{h \times h}$: Matriz de pesos para la transición recurrente de estados.
- $W_{hq} \in \mathbb{R}^{h \times q}$: Matriz de pesos para la capa de salida.
- $\phi(\cdot)$: Función de activación no lineal (típicamente tanh o ReLU).

Compartición de Pesos (*Weight Sharing* Temporal): Las matrices de pesos $\{W_{xh}, W_{hh}, W_{hq}\}$ son exactamente las mismas en todos los pasos de tiempo $t = 1, \ldots, T$.

```
Estructura Desplegada en el Tiempo (Unfolded RNN):


  y_1          y_2                  y_T


 h_1       h_2      ...      h_T

     W_hh W_hh


     W_xh         W_xh                 W_xh

  x_1          x_2                  x_T
```

1.3 3. Entrenamiento: Retropropagación a través del Tiempo (BPTT)

1.3.1 3.1 Backpropagation Through Time (BPTT)

Para entrenar las RNNs, la red recurrente se "despliega" a lo largo de los T pasos temporales formando un grafo acíclico dirigido. La pérdida total es la suma de las pérdidas en cada paso de tiempo:

$$\mathcal{L}(W) = \sum_{t=1}^{T} \mathcal{L}_t(y_t, \hat{y}_t)$$

Para calcular el gradiente respecto a la matriz de transición recurrente W_{hh}, se aplica la regla de la cadena acumulando las dependencias pasadas:

$$\frac{\partial \mathcal{L}_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial h_t} \cdot \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_k}{\partial W_{hh}}$$

1.3.2 3.2 Problema Crítico: Desvanecimiento y Explosión del Gradiente

El término $\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$ implica multiplicar repetidamente la matriz de pesos $W_{hh}^{\top}$.

- **Desvanecimiento del Gradiente (*Vanishing Gradients*): **Si los autovalores de W_{hh} son menores a 1, la magnitud del gradiente decrece exponencialmente hacia cero cuando la secuencia es larga ($T \gg 0$). La red** olvida las dependencias a largo plazo**.

- **Explosión del Gradiente (*Exploding Gradients*): **Si los autovalores son mayores a 1, el gradiente crece exponencialmente destruyendo el entrenamiento. Se soluciona mediante** Gradient Clipping**:

$$g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) g$$

1.4 4. Redes Gated: LSTM y GRU

Para solucionar el desvanecimiento de gradiente y permitir recordar información a largo plazo, se crearon arquitecturas basadas en puertas de control (*gates*).

1.4.1 4.1 LSTM (*Long Short-Term Memory*)

La LSTM sustituye la celda estándar por una estructura de memoria que mantiene un **estado de celda** C_t (*Cell State* o autopista de información) y un **estado oculto** h_t.

```
                                        (Cell State C_t)

                                  x (f_t)        + (i_t * C~_t)

C_{t-1}                                            C_t
                                                                 
                                                                 
                                                                 
                                                                 
h_{t-1}                                            h_t
                                                                 
                                                                 
x_t      [Forget Gate]    [Input Gate] [Candidate] [Output Gate]
```

1.4.2 Puertas y Ecuaciones Matemáticas de la LSTM:

1. **Forget Gate (f_t):** Decide qué información descartar del estado de celda anterior.

$$f_t = \sigma\left(x_t W_{xf} + h_{t-1} W_{hf} + b_f\right)$$

1. **Input Gate (i_t) y Candidato ($\tilde{C}_t$):** Deciden qué nueva información almacenar.

$$i_t = \sigma\left(x_t W_{xi} + h_{t-1} W_{hi} + b_i\right)$$

$$\tilde{C}_t = \tanh\left(x_t W_{xc} + h_{t-1} W_{hc} + b_c\right)$$

1. **Actualización del Cell State (C_t):**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

1. **Output Gate (o_t) y Estado Oculto (h_t):**

$$o_t = \sigma\left(x_t W_{xo} + h_{t-1} W_{ho} + b_o\right)$$

$$h_t = o_t \odot \tanh(C_t)$$

1.4.3 4.2 GRU (*Gated Recurrent Unit*)

Variante simplificada de la LSTM que combina el estado de celda y el estado oculto en una sola variable h_t, reduciendo el coste computacional y el número de parámetros.

1. **Reset Gate (r_t):** Determina cuánta información del pasado olvidar:

$$r_t = \sigma\left(x_t W_{xr} + h_{t-1} W_{hr} + b_r\right)$$

1. **Update Gate (z_t):** Actúa simultáneamente como puerta de olvida e entrada:

$$z_t = \sigma\left(x_t W_{xz} + h_{t-1} W_{hz} + b_z\right)$$

1. **Candidato de Estado Oculto ($\tilde{h}_t$):**

$$\tilde{h}_t = \tanh\left(x_t W_{xh} + (r_t \odot h_{t-1}) W_{hh} + b_h\right)$$

1. **Estado Oculto Final (h_t):**

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

1.5 5. RNNs Bidireccionales y Apiladas (*Deep RNNs*)

- **RNN Bidireccional (BiRNN / BiLSTM):** Procesa la secuencia en ambas direcciones simultáneamente usando dos estados ocultos independientes ($\overrightarrow{h}_t$ hacia adelante y $\overleftarrow{h}_t$ hacia atrás). El estado combinado es $h_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t]$.

- *Uso:* Crucial en tareas de procesamiento de lenguaje natural (NLP) donde el contexto futuro influye en el significado del presente.

- **RNN Profunda / Apilada (*Stacked RNN*):** Apila múltiples capas recurrentes donde la salida del estado oculto de la capa l sirve como entrada x_t para la capa $l + 1$.

1.6 6. Práctica y Aplicación en Código (PyTorch)

A continuación se incluye una implementación modular completa que ilustra la construcción de una red **LSTM Bidireccional** para la clasificación de secuencias (ej. análisis de sentimiento o series temporales) en PyTorch:

```

import torch

import torch.nn as nn

import torch.optim as optim



# ------------------------------------------------------------------------

# 1. Definición del Modelo BiLSTM / GRU en PyTorch

# ------------------------------------------------------------------------

class SequenceClassifier(nn.Module):

    def __init__(self, vocab_size=1000, embed_dim=64, hidden_dim=128, num_classes=2, num_layers=2, dropout=0.3):

        super(SequenceClassifier, self).__init__()



        # Capa de Embedding para convertir índices de tokens en vectores continuos

        self.embedding = nn.Embedding(vocab_size, embed_dim)



        # Capa LSTM Bidireccional de múltiples capas

        self.lstm = nn.LSTM(

            input_size=embed_dim,

            hidden_size=hidden_dim,

            num_layers=num_layers,

            batch_first=True,       # Entrada esperada: (Batch, Seq_Len, Embed_Dim)

            bidirectional=True,     # BiLSTM

            dropout=dropout if num_layers > 1 else 0.0

        )



        # Capa lineal final: hidden_dim * 2 por la bidireccionalidad

```

```
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

        self.dropout = nn.Dropout(p=dropout)



    def forward(self, x):

        # x shape: (Batch, Seq_Len)

        embedded = self.embedding(x) # (Batch, Seq_Len, Embed_Dim)



        # output shape: (Batch, Seq_Len, Hidden_Dim * 2)

        # h_n, c_n shape: (Num_Layers * 2, Batch, Hidden_Dim)

        lstm_out, (h_n, c_n) = self.lstm(embedded)



        # Concatenar el último estado oculto forward y backward de la última capa

        hidden_forward = h_n[-2, :, :]

        hidden_backward = h_n[-1, :, :]

        last_hidden = torch.cat((hidden_forward, hidden_backward), dim=1) # (Batch, Hidden_Dim * 2)



        out = self.dropout(last_hidden)

        logits = self.fc(out) # (Batch, Num_Classes)

        return logits



# ------------------------------------------------------------------------

# 2. Ejemplo de Bucle de Entrenamiento

# ------------------------------------------------------------------------

def train_rnn_demo():

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")



    # Hiperparámetros

    VOCAB_SIZE = 1000

    BATCH_SIZE = 16
```

```
    SEQ_LEN = 32

    NUM_CLASSES = 2


    # Modelo, pérdida y optimizador

    model = SequenceClassifier(

        vocab_size=VOCAB_SIZE,

        embed_dim=64,

        hidden_dim=128,

        num_classes=NUM_CLASSES,

        num_layers=2

    ).to(device)


    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(model.parameters(), lr=0.001)


    # Datos sintéticos de secuencia: (Batch, Seq_Len)

    dummy_input = torch.randint(0, VOCAB_SIZE, (BATCH_SIZE, SEQ_LEN)).to(device)

    dummy_labels = torch.randint(0, NUM_CLASSES, (BATCH_SIZE,)).to(device)


    model.train()

    optimizer.zero_grad()


    # Forward Pass

    logits = model(dummy_input)

    loss = criterion(logits, dummy_labels)


    # Backward Pass

    loss.backward()

```

```
    # Gradient Clipping esencial para evitar la explosión de gradiente en RNNs
    nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)


    optimizer.step()


    print(f"Demostración RNN completada en dispositivo: {device}")
    print(f"Dimensión de salida (Logits): {logits.shape}")
    print(f"Pérdida inicial (Loss): {loss.item():.4f}")


if __name__ == "__main__":
    train_rnn_demo()
```

Dime si necesitas que profundicemos en algún aspecto de las **RNNs, LSTMs o GRUs** o si te gustaría preparar un cuestionario interactivo para poner a prueba estos conceptos de cara a tu máster.